Module-1 Assignment

Part A: Theoretical Questions

1. **Explain what DevOps Engineering is. How does it differ from traditional development and operations roles?**

   Answer:

   DevOps Engineering is a bridge practice, culture, and set of technical tools that unites software development (Dev) and IT operations (Ops) into a single, automated, and continuous lifecycle. Instead of treating building and running software as separate steps, a DevOps engineer builds the pipelines, cloud infrastructure, and monitoring tools that let developers ship code safely, quickly, and repeatedly.

   ## Key Differences: DevOps vs. Traditional Development and Operations

   In a traditional setup, development and operations function as isolated silos. In a DevOps model, those walls are torn down.

Feature	**Traditional Development & Operations**	**DevOps Engineering**
Team Structure	Siloed; developers write code and "throw it over the wall" to operations.	Cross-functional or unified teams sharing end-to-end responsibility.
Workflow Approach	Sequential (Waterfall); large, infrequent, and risky releases (weeks or months).	Iterative; small, frequent, and automated releases (daily or hourly).
Infrastructure Management	Manual server configuration, static hardware, and slow ticketing systems.	**Infrastructure as Code (IaC)** using declarative scripts (e.g., Terraform, Ansible).
Deployment Method	Manual handoffs, high risk of human error, and prolonged downtime.	**CI/CD Pipelines** (Continuous Integration/Continuous Delivery) for automated testing and rollout.
Culture & Failure	Blame game; operations fears change, developers push for speed.	Shared ownership; "fail-early" mindset with automated testing and quick feedback loops.

2. Describe how DevOps fits into the Software Development Life Cycle (SDLC). Mention at least three SDLC phases where DevOps adds value. What values does a DevOps Engineer bring to an organization? Explain any four values with examples.

Answer:

DevOps is not a separate phase of the Software Development Life Cycle (SDLC); rather, it is a cultural shift, set of practices, and automation toolkit that integrates development (Dev) and operations (Ops) across the entire lifecycle. It removes traditional silos to enable continuous, faster, and more reliable software delivery.

Here is how DevOps adds distinct value across three key SDLC phases:

- **Design & Planning:** DevOps introduces shift-left testing and architecture. By involving operations early, teams design systems with deployment, scalability, and monitoring in mind, preventing architectural bottlenecks later.
- **Testing & Building:** DevOps leverages **Continuous Integration (CI)**. Every code change automatically triggers automated builds and test suites, catching bugs immediately rather than right before a major release.
- **Deployment & Operations:** DevOps uses **Continuous Delivery (CD)** and Infrastructure as Code (IaC). Environments are provisioned automatically, and deployments are executed seamlessly via automated pipelines, reducing downtime and human error.

4 Core Values a DevOps Engineer Brings to an Organization

A DevOps Engineer acts as the bridge between code creation and system stability. Four key values they bring include:

1. Faster Time-to-Market

DevOps Engineers eliminate manual bottlenecks by building automated CI/CD pipelines. This allows businesses to release features and fixes to customers at a rapid pace.

- *Example:* Instead of a software team manually bundling code, running scripts, and deploying updates once a month, a DevOps Engineer automates the pipeline so that code updates are automatically tested and **deployed to production multiple times a day** with a single click.

2. Enhanced Infrastructure Scalability and Reliability

Through **Infrastructure as Code (IaC)**, DevOps Engineers treat infrastructure setups (servers, networks, databases) exactly like software code. This makes environment provisioning consistent, repeatable, and easily scalable.

- *Example:* If a sudden spike in user traffic occurs, a DevOps Engineer's pre-configured cloud scripts (like Terraform or Kubernetes) will **automatically spin up extra server instances** to handle the load and spin them down afterward to save costs.

3. Reduced Human Error and Higher Quality

By automating repetitive tasks—such as code linting, security scanning, and regression testing—DevOps Engineers minimize the risk of human oversight, ensuring higher code quality before it ever reaches the user.

- *Example:* A DevOps Engineer configures the pipeline to block any deployment if the code fails security vulnerability scans or automated unit tests, **preventing buggy or insecure code from reaching live users**.

4. Improved Collaboration and Shared Responsibility

DevOps Engineers break down the traditional "us vs. them" wall between developers (who want to ship features fast) and operations teams (who want to keep systems stable). They align both teams around shared metrics and tools.

- *Example:* A DevOps Engineer sets up a **centralized observability dashboard** (using tools like Grafana or Datadog). Both developers and system administrators look at the exact same data to troubleshoot an outage together, rather than playing the blame game.

3.Why is prior Software Engineering (SWE) experience considered helpful before becoming a DevOps Engineer?

Answer:

Prior Software Engineering (SWE) experience is helpful for a DevOps Engineer because it provides a deep, practical understanding of how applications are built, tested, and deployed from the developer's perspective.

Key Reasons Why SWE Experience Helps in DevOps

- **Empathy for Developers:** Knowing the pain points of writing code, managing local environments, and dealing with broken build pipelines allows DevOps engineers to build better, developer-friendly automation.
- **Strong Coding and Scripting Skills:** Writing robust Infrastructure as Code (IaC) with tools like Terraform, or automation scripts in Python, Ruby, or Bash, comes naturally to those with a programming background.
- **Understanding the SDLC (Software Development Life Cycle):** SWEs already understand version control (Git), code reviews, branching strategies, and unit testing, which are foundational to designing CI/CD pipelines (using GitHub Actions, Jenkins, etc.).
- **Better Troubleshooting and Debugging:** When an application crashes in production, a DevOps engineer with a development background can read the stack trace, understand the application logic, and collaborate effectively with software teams to fix the root cause rather than just rebooting a server.
- **Architecture and System Awareness:** Developers learn how microservices, APIs, and databases interact, making it easier to design scalable cloud environments (AWS, GCP, Azure) and containerized systems with Docker and Kubernetes.

4.Explain why networking knowledge (DNS, ports, protocols, IP addressing) is important for DevOps roles.

Answer:

Networking knowledge—specifically **DNS**, **ports**, **protocols**, and **IP addressing**—is the backbone of building, deploying, securing, and troubleshooting modern cloud and containerized infrastructure.

1. IP Addressing (Public vs. Private & CIDR/Subnetting)

- **Why it matters:** An IP address uniquely identifies a machine, container, or virtual server on a network.
- **DevOps Use Case:**
    - **Cloud Provisioning (IaC):** When writing Terraform or cloud templates, you must design Virtual Private Clouds (VPCs), subnets, and routing tables using **CIDR notation** (e.g., /16 or /24) to properly isolate public and private tiers.
    - **Traffic Routing:** Separating **private IPs** (for internal databases and backend microservices) from **public IPs** (for load balancers or gateways) prevents accidental exposure of sensitive data to the internet.

2. DNS (Domain Name System)

- **Why it matters:** DNS translates human-readable domain names (like ://company.com) into numeric IP addresses.
- **DevOps Use Case:**
    - **Service Discovery:** In dynamic environments like Kubernetes or microservice architectures, containers scale up and down with changing IPs. DNS and internal service discovery allow components to find and talk to each other reliably without hardcoding static IP addresses.
    - **Traffic Management & Routing:** Managing DNS records (A, CNAME, TXT) is essential for zero-downtime deployments, blue-green deployments, canary releases, and content delivery networks (CDNs).
    - **Debugging:** "The website won't load" is often a DNS propagation, TTL (Time-to-Live), or internal resolver misconfiguration rather than an application crash.

3. Ports

- **Why it matters:** If an IP address is like the street address of a building, a port is the specific room number that routes traffic to a distinct application running inside that server.
- **DevOps Use Case:**
    - **Service Binding:** Knowing that **SSH** runs on port 22, **HTTP** on port 80, **HTTPS** on port 443, or **MySQL** on port 3306 is mandatory when configuring server software.
    - **Container Mapping:** In Docker and Kubernetes, you must map container ports to host ports (e.g., -p 8080:80) so external traffic can reach the app.
    - **Firewalls & Security Groups:** Writing strict security group rules, Network ACLs, or iptables requires specifying exact open or closed ports to lock down attack surfaces.

4. Protocols (TCP, UDP, HTTP/HTTPS, gRPC, SSH)

- **Why it matters:** Protocols define the shared rules, handshakes, and data-formatting standards for how packets are transmitted across a network.
- **DevOps Use Case:**
    - **Reliability vs. Speed:** Knowing when to use **TCP** (reliable, 3-way handshake used for web and APIs) versus **UDP** (fast, connectionless used for streaming or metrics) prevents performance bottlenecks.
    - **Security & Encryption:** Enforcing **HTTPS/TLS** certificates ensures that traffic passing through CI/CD pipelines, APIs, and cloud load balancers is encrypted.
    - **Automation & Access:** Tools like **SSH** power remote server administration and configuration management (e.g., Ansible), while modern microservices lean on **gRPC** or WebSocket's for high-performance internal communication.

How This Ties Together in Troubleshooting

When an application fails in production, a DevOps engineer relies on networking fundamentals and command-line tools—such as ping, curl, dig/nslookup, netstat/ss, and traceroute—to isolate the failure layer instantly:

- A **"Connection Refused"** error means the server answered, but no service was listening on that port.
- A **Timeout** means a firewall, security group, or routing table is dropping the packets before they reach the destination

5.What soft skills are essential for a DevOps Engineer? Explain how these skills help during incidents or failures.

Answer:

Essential soft skills for a DevOps engineer include communication, calm decision-making (adaptability under pressure), empathy and collaboration, documentation, and conflict resolution (blameless culture).

Essential Soft Skills & Their Role During Incidents or Failures

- **Communication**
    - *What it is:* Writing clear incident notes, posting concise status updates, and explaining technical states plainly to both engineers and non-technical stakeholders.
    - *During incidents:* Prevents panic, stops people from asking "what is going on?" every two minutes, and ensures leadership and customer support teams have accurate information to share.
- **Calm Decision-Making & Adaptability**
    - *What it is:* Staying level-headed in high-stress moments, knowing when to say *"I don't know yet,"* and deciding when to roll back changes versus hotfix.

    - *During incidents:* Stops engineers from making rushed, reckless edits that make an outage worse. It allows the team to methodically isolate the root cause instead of guessing.

- **Empathy & Cross-Functional Collaboration**
    - *What it is:* Understanding the pressures and workflows of developers, security, and product teams rather than working in an isolated silo.
    - *During incidents:* Keeps teams united. Instead of developer teams blaming operations or security pointing fingers at code, empathy helps everyone work toward the shared goal of restoring service.
- **Documentation**
    - *What it is:* Maintaining clean, updated runbooks, architecture maps, and known gotchas.
    - *During incidents:* Serves as your "invisible voice". When a critical system breaks at 2:00 AM, a well-written runbook or incident guide lets whoever is on-call resolve the issue quickly without hunting down the original creator of the system.
- **Conflict Resolution & Blameless Culture**
    - *What it is:* Shifting post-incident conversations away from *"who caused this?"* to *"how do we fix this and prevent it?"*
    - *During incidents (and post-mortems):* Creates **psychological safety**. When engineers are not afraid of punishment or blame, they report near-misses faster, share honest root causes, and build stronger, more resilient systems moving forward.

6.Explain the importance of patience and troubleshooting mindset in DevOps with a real-life example.

Answer:

Patience and a troubleshooting mindset prevent rushed, destructive decisions during live system incidents, allowing engineers to methodically locate root causes instead of applying temporary, error-prone patches.

Why Patience and a Troubleshooting Mindset Matter in DevOps

- **Avoids Panic-Driven Mistakes:** Rushing to restart services or push blind hotfixes under pressure often masks the real symptom and triggers secondary outages.
- **System-Wide Perspective:** A true troubleshooting mindset looks across complex layers—such as cloud networking, IAM permissions, Kubernetes pods, and database connections—rather than blaming a single component.
- **Data-Driven Diagnostics:** Relying on metrics, distributed traces, and log aggregation (like Prometheus, Grafana, or ELK) ensures decisions are built on facts rather than guesswork.
- **Long-Term Resilience:** Patient investigation leads to thorough post-mortems, turning live failures into automated guardrails and stronger fail-safe mechanisms.

Real-Life Example: The Sudden E-Commerce Latency Spike

The Scenario

An e-commerce platform experiences a massive latency spike and intermittent HTTP 5xx errors during a major flash sale. Customer checkouts are timing out, and revenue is dropping every second.

The Rushed (Flawed) Approach

- An engineer panics and immediately doubles the number of backend Kubernetes pods and restarts the primary database cluster.
- **The Result:** The database restart locks all active transactions, causing a complete site-wide outage instead of a slow response.

The Patient, Troubleshooting Mindset Approach

1. **Observe and Gather Data:** Instead of touching anything, the engineer checks the observability dashboard (Grafana/Prometheus) and notices that CPU and memory are normal, but database connection pool wait times have spiked.
2. **Trace the Root Cause:** They inspect application logs and database query metrics using tools like the Dev Community Troubleshooting Guide. They discover a newly deployed microservice is running an unindexed query on the inventory table repeatedly.
3. **Isolate and Resolve:** Rather than crashing the database, the engineer throttles traffic to that specific non-critical recommendation endpoint or applies a quick feature-flag kill switch to disable the bad query.
4. **Permanent Fix:** After the sale calms down, the team writes a proper database index, updates the CI/CD test suite to catch unindexed queries in staging, and documents the incident in a blame-free post-mortem.

Part B: Scenario-Based Questions

1. A new student joins a DevOps course but is unfamiliar with GitHub and cloud platforms. How would you onboard this student efficiently using tools, documentation, and communication channels?

Answer:

To onboard a student unfamiliar with GitHub and cloud platforms efficiently, you should use a structured, scaffolded approach that minimizes tool fatigue and prioritizes practical, hands-on wins.

Here is an efficient onboarding framework broken down by communication, documentation, and tools.

1. Communication & The First 48 Hours

The goal of initial communication is to reduce anxiety and establish a clear path forward.

- **Welcome Message:** Send a concise, automated but warm welcome email. Include a **single tracking link** to the onboarding checklist so they aren't overwhelmed by multiple URLs.
- **1-on-1 Sync (15 Mins):** Schedule a brief video call to help them set up their local environment (e.g., installing VS Code and Git) if they get stuck.
- **Dedicated Slack/Discord Channel:** Add them to a specific #help-newbies or #devops-onboarding channel. Introduce them to a **peer mentor** (an advanced student or teaching assistant) so they always have a safe space to ask "basic" questions.

2. Scaffolded Documentation (The "Learn by Doing" Path)

Do not hand the student official AWS or GitHub documentation on day one; it is too dense. Instead, provide a curated, three-tiered documentation path:

- **Phase 1: Concepts via Analogies**
    - *GitHub:* Explain it as a "Google Doc for code with a time machine."
    - *The Cloud:* Explain it as "renting someone else's massive computer instead of buying your own."
- **Phase 2: The "Golden Path" Runbooks**
    - Provide step-by-step, screenshot-heavy internal documentation.

    - Limit the first guide to exactly **three outcomes**: Creating a GitHub account, cloning a repository, and pushing a single text file change.

- **Phase 3: Sandbox Environment**
    - Provide documentation on how to access a **pre-configured sandbox** (e.g., GitHub Codespaces or a restricted AWS Academy account) so they cannot break production infrastructure or incur high costs while learning.

3. Tooling & Automation Setup

Leverage tools that abstract away early complexities so the student can focus on DevOps concepts rather than syntax errors.

Category	**Tool**	**Purpose for Beginners**
Code Editor	VS Code + GitLens	Avoids the command line early on. GitLens provides a visual way to see code history and stage changes.
Cloud Sandbox	GitHub Codespaces	Eliminates local environment setup ("it works on my machine" issues) by providing a browser-based dev environment.
Interactive Learning	GitHub Skills	Free, interactive courses natively built into GitHub that teach Git basics through automated pull request reviews.
Cloud Learning	AWS Academy / GCP Edu	Provides free, isolated lab environments with zero risk of unexpected billing.

4. The 5-Day Onboarding Roadmap

- **Day 1: Git Basics.** Install VS Code and Git. Complete the "Introduction to GitHub" interactive tutorial.
- **Day 2: First Commit.** Clone the course repository, create a branch named student-intro, add a bio file, and open a Pull Request.

- **Day 3: Cloud Conceptual.** Log into the cloud sandbox environment. Navigate the console GUI to manually launch a single virtual machine (e.g., EC2 instance).
- **Day 4: Connecting the Dots.** Learn how the cloud and GitHub talk to each other (e.g., deploying a simple HTML page using GitHub Actions).
- **Day 5: Check-in.** Peer mentor reviews the student's progress and clears up any lingering conceptual blocks.

2.During a production deployment, an application goes down unexpectedly. As a DevOps Engineer, describe step-by-step how you would troubleshoot and handle the situation.

Answer:

To handle a sudden production application outage during a deployment, follow a calm, evidence-based framework that prioritizes service recovery first and root-cause analysis second.

Step 1: Confirm the Impact and Scope

- **Verify availability**: Check the public-facing URL, internal health-check endpoints, and load balancer target health (such as AWS ALB unhealthy targets or Kubernetes pod readiness).
- **Define the blast radius**: Determine if the failure affects all users globally, a specific region, a single node, or a subset of routes.
- **Acknowledge and communicate**: Post an initial update in the incident response channel (Slack/Teams) and alert relevant stakeholders or support teams to avoid duplicate troubleshooting.

Step 2: Check Recent Changes (The Deployment)

- **Inspect the deployment pipeline**: Check CI/CD logs (Jenkins, GitHub Actions, GitLab CI) to see what commit, tag, or container image was just rolled out.
- **Review config and secrets**: Look for recent environment variable changes, feature flags, or modified configuration and secret files.
- **Check infrastructure drift**: Verify if any infrastructure-as-code changes (Terraform/Ansible) were applied alongside the code release.

Step 3: Check Monitoring and Metrics

- **Inspect golden signals**: Open your observability platform (Datadog, Prometheus, Grafana, or CloudWatch) to review traffic, error rates, latency, and resource saturation.
- **Look for resource exhaustion**: Check if the app crashed due to CPU throttling, memory leaks, OOMKilled (out-of-memory) errors, or a full disk from log accumulation.
- **Trace the request path**: Follow the flow layer-by-layer—from DNS and Ingress/Load Balancer down to the service mesh, pods/containers, and backend databases.

Step 4: Check Application and System Logs

- **Tail recent logs**: Look at application logs, container standard output, or cluster events (such as Kubernetes CrashLoopBackOff or probe failures).
- **Search for critical errors**: Identify stack traces, database connection pool timeouts, unhandled exceptions, or authentication/token failures around the exact timestamp of the deployment.

Step 5: Stabilize the Service First (Mitigation)

- **Execute an immediate rollback**: If the new release is broken and a fix is not instantly obvious, trigger an automated or manual rollback to the last known good version or artifact.
- **Scale or restart if safe**: If it is a transient connection drop or resource lock, scale up replicas or gracefully restart pods/services *only* if logs confirm it will not corrupt state.
- **Verify recovery**: Confirm that health checks turn green and live user traffic flows normally again.

Step 6: Investigate Root Cause and Prevent Recurrence

- **Perform a deep-dive analysis**: Once the system is stable, analyze why the deployment broke production (e.g., missing database migration, incompatible library version, or misconfigured environment variable).
- **Implement safeguards**: Add missing health/readiness probes, adjust resource limits, update automated tests, or switch to canary/blue-green deployment strategies to catch the issue before it hits production next time.
- **Conduct a blameless postmortem**: Document the timeline, impact, root cause, and action items in an incident report to share with the engineering team.

3. Your team releases features frequently, but system stability is decreasing. How can DevOps practices help balance speed and reliability?

Answer:

DevOps practices balance speed and stability by replacing manual handoffs and unvalidated changes with continuous automation, early testing, and shared operational responsibility.

Key DevOps Practices to Restore Stability

- **Continuous Integration and Continuous Delivery (CI/CD):** Automatically merge, build, and test every code commit using pipelines from providers like GitHub or Atlassian. Small, frequent releases limit the impact of failures and make troubleshooting straightforward.
- **Shift-Left Testing:** Move quality assurance and security checks earlier in the development lifecycle rather than waiting until the end. Automated unit, integration, and security tests catch bugs before they reach production.
- **Infrastructure as Code (IaC) and Containerization:** Provision and configure servers, networks, and environments programmatically using tools like Terraform or Ansible. This ensures development, staging, and production environments remain identical, eliminating "it works on my machine" failures.
- **Continuous Monitoring and Observability:** Aggregate logs, metrics, and traces in real time using platforms like Prometheus and Grafana. Early detection alerts teams to performance degradation before users notice.
- **Automated Rollbacks and Feature Flags:** Decouple deployment from feature release. Use feature flags to turn risky features on or off dynamically, and configure automated rollbacks to revert to a stable state if a health check fails.
- **Shared Ownership and Blameless Culture:** Break down silos between development and operations teams. When developers share responsibility for production performance, code quality and operational resilience naturally improve.

4. A team struggles with communication during incidents because members are distributed. How can proper onboarding to communication platforms improve DevOps workflows?

Answer:

Proper onboarding to communication and ChatOps platforms cuts down the "coordination tax" in distributed teams by establishing instant clarity, reducing Mean Time to Resolution (MTTR), and preventing panic during outages

How Proper Platform Onboarding Improves DevOps Workflows

- **Standardized Channel Architecture:** Onboarding ensures every team member knows where to go the moment an incident is declared (e.g., dedicated emergency response channels like a temporary Slack "war room") instead of scattering discussions across direct messages or fragmented emails.
- **ChatOps Integration and Automation:** Training teaches engineers how to trigger runbooks, pull real-time telemetry from tools like Prometheus or Datadog, and deploy rollbacks directly inside the chat interface via tools like Rootly or PagerDuty, eliminating context-switching across ten different dashboards.
- **Defined Roles and Escalation Paths:** Onboarding clarifies who acts as the Incident Commander, who manages technical mitigation, and who handles stakeholder communications, preventing chaotic "too many cooks in the kitchen" scenarios.
- **Asynchronous & Cross-Time-Zone Hand-offs:** For distributed "follow-the-sun" teams, proper training on thread management, status update templates, and chronological logging ensures that incoming shifts inherit full context without playing catch-up.
- **Blameless Postmortem Culture:** Structured onboarding reinforces that communication during and after an incident focuses strictly on objective data and system timelines rather than finger-pointing, leading to faster root-cause learning.

Part C: Technical / Practical Tasks

Task 1 Solution:

To get a local DevOps demo running, we will use a Node.js and Express web server. Setting up an application locally allows you to mimic production environments, verify code changes instantly, and build automated deployment pipelines.

Here is your quick-start guide to running a local Express application, followed by how this fits into DevOps workflows.

Step-by-Step: Running a Local Express App

1. Initialize the Project

Open your terminal, create a new directory, and initialize a Node.js project:

mkdir local-devops-demo

cd local-devops-demo

npm init -y

2. Install Express

Install the Express framework as a dependency:

npm install express

3. Create the Router Code

Create a file named app.js and paste the following basic web server code:

```
import express, { Request, Response } from "express";

const app = express();

// Sample application route

app.use(express.json());

app.get("/", (req: Request, res: Response) => {

  res.send('<h1>Welcome!</h1><p> On behalf of the entire team, I would like to extend our warmest wishes on first Assignment.</p>');

 });

export default app;
```

3. Create the Server Code

Create a file named index.js and paste the following basic web server code:

```
import app from "./app";

const PORT = process.env.PORT || 3000;

app.listen(PORT, () => {
  console.log(`Server running at http://localhost:${PORT}`);
});
```

4. Run the Application

Start your server by executing:

```
npm run dev
```

You should see the message: Server running at http://localhost:3000 in your terminal.

5. Verify the Execution

Open your web browser or use a terminal tool to access the URL:

http://localhost:3000

In browser show below message:

Welcome!

On behalf of the entire team, I would like to extend our warmest wishes on first Assignment.

How Local Execution Powers DevOps Workflows

Running an Express application locally is the foundation of modern DevOps practices because it bridges the gap between writing code and shipping it to production.

• Rapid Feedback Loops: Developers can use tools like nodemon alongside Express to instantly reload the app when code changes. This avoids waiting for a full cloud deployment cycle just to test a single line of code.

• Shift-Left Testing: By catching bugs, syntax errors, or broken dependencies locally on http://localhost:3000, you prevent broken code from ever reaching the shared CI/CD pipeline or staging environments.

• Pre-Flight Configuration Testing: You can test how the application handles environment variables (like changing process.env.PORT) locally before injecting production secrets into live environments.

Task 2 Solution:

I can complete both tasks on Windows computer using a simple nodejs server and ngrok. This approach is beginner-friendly and does not require deploying an application to a production server.

Start a web application locally and verify that it works.

Open Command Prompt (cmd) and run:

npm run dev

Open browser and visit:

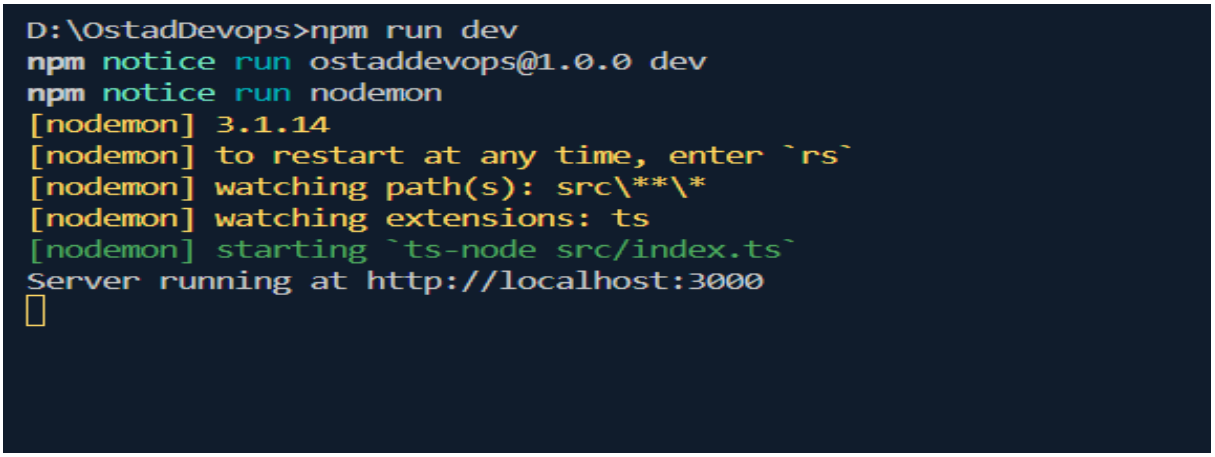
http://localhost:3000

Expected result: Browser displays “Welcome!

On behalf of the entire team, I would like to extend our warmest wishes on first Assignment.” and Terminal show “Server running at http://localhost:3000”

Screenshot 1 — Local application

Show your browser displaying the demo page at http://localhost:3000.



Expose local application using ngrok:

Local web server — port 3000

ngrok tunnel

ngrok http 3000

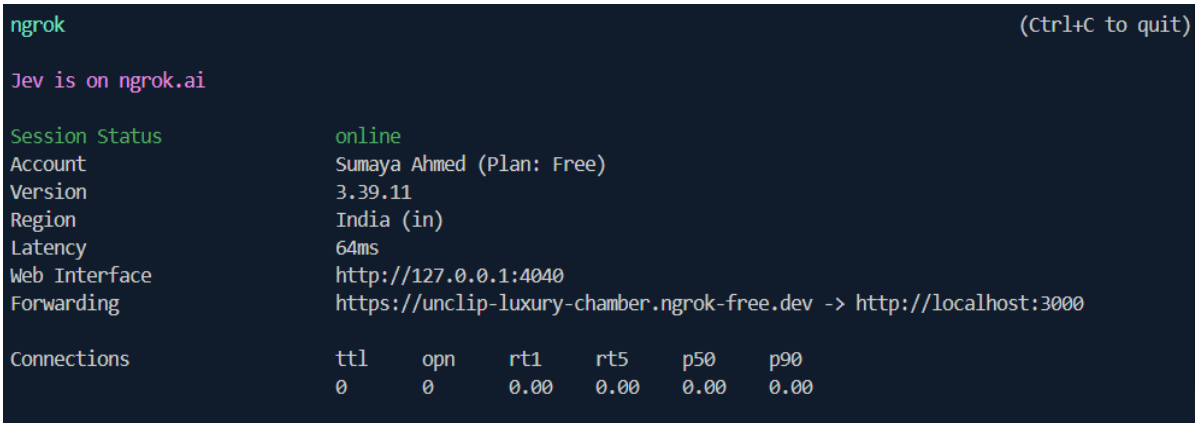
Public internet URL:

Web Interface http://127.0.0.1:4040

Forwarding https://unclip-luxury-chamber.ngrok-free.dev -> http://localhost:3000

Screenshot 2 — Public tunnel

Show the ngrok terminal with the actual HTTPS forwarding URL and the local port 3000.



Real-world use case:

ngrok allows developers to share a locally running application with clients or teammates for demonstrations and testing before deploying it to production.

Evidence: Screenshot of the ngrok terminal showing the forwarding URL.